

第五章 树与二叉树

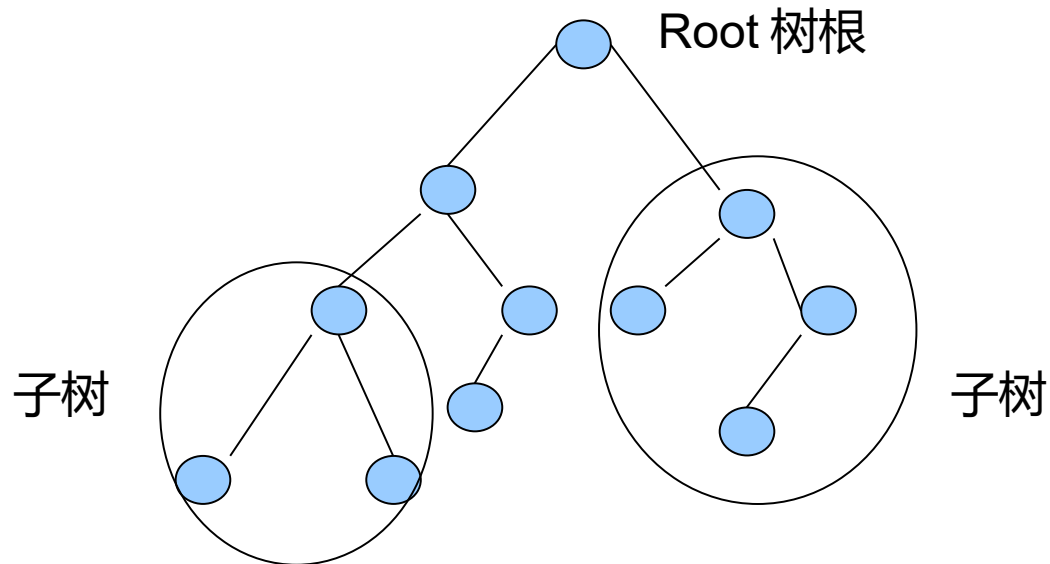
第5章 树与二叉树

- 5.1 树的基本概念
- 5.2 二叉树
- 5.3 二叉树的遍历
- 5.5 线索二叉树
- 5.5 树和森林
- 5.6 哈夫曼树

5.1 树的基本概念

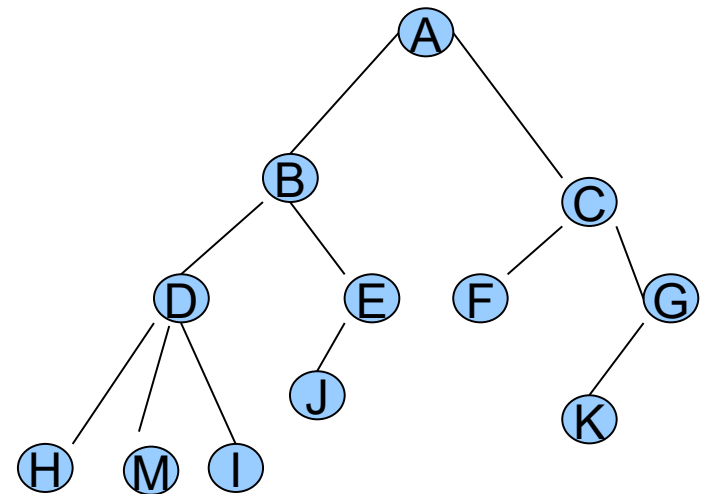
5.1.1 树的定义及表示

树 : n 个结点的有限集 ($n \geq 0$)



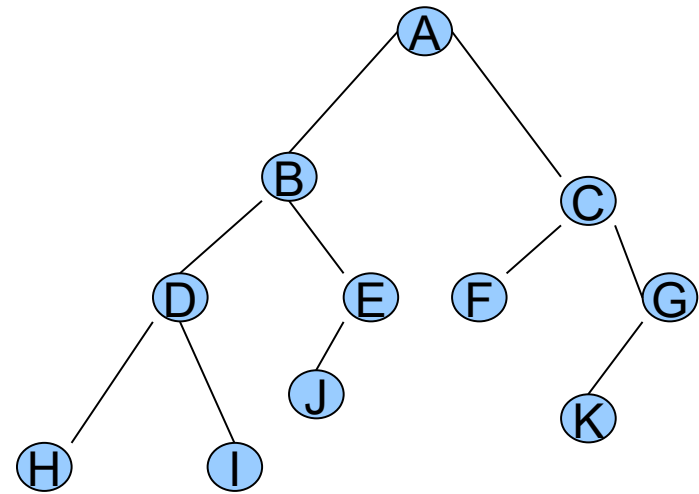
5.1.2 树的常用术语及运算

- 树的结点
- 结点的度 --- 结点拥有的子树的个数
- 叶子结点 --- 度为 0 的结点, 又称终端结点
- 分枝结点 --- 度不为 0 的结点
- 树的度 --- 树内结点度的最大值
- 树的层次 --- 第一层从根结点开始, 根的孩子在第二层, 依此类推
- 树的深度 --- 树中结点的最大层次



5.1.2 树的常用术语及运算

- 孩子 (结点) --- 结点的子树的根结点
- 双亲 (结点) --- 结点作为根结点的子树的根结点
- 兄弟 (结点) --- 同一个双亲的孩子结点之间互为兄弟
- 祖先 --- 从根结点到该结点所经分枝上的所有结点
- 子孙 --- 以某结点为根的子树中的任一结点都是该结点的子孙



5.1.2 树的常用术语及运算

- 树的基本运算

- (1) $\text{setnull}(T)$: 初始化操作, 置 T 为空树。
- (2) 求根结点 $\text{ROOT}(T)$ 或 $\text{ROOT}(x)$ 。求树 T 的根或求结点 x 所在的树的根结点。若 T 是空树或 x 不在任何一棵树上, 则函数值为“空”。
- (3) 求双亲结点 $\text{RARENT}(T,x)$ 。求树 T 中结点 x 的双亲结点。若结点 x 是树 T 的根结点或结点 x 不在树 T 中, 则函数值为“空”。
- (5) 求孩子结点 $\text{CHILD}(T,x,i)$ 。求树 T 中结点 x 的第 i 个孩子结点。若结点 x 是树 T 的叶子或无第 i 个孩子或结点 x 不在树 T 中, 则函数值为“空”。
- (5) 建树 $\text{creat}(x,F)$ 。生成以 x 结点为根、以森林 F 为子树森林的树。
- (6) 求右兄弟结点 $\text{RIGHT_SIBLING}(T,x)$ 。求树 T 中结点 x 右边的兄弟。若结点 x 是其双亲的最右边的孩子结点或结点 x 不在树 T 中, 则函数值为“空”。

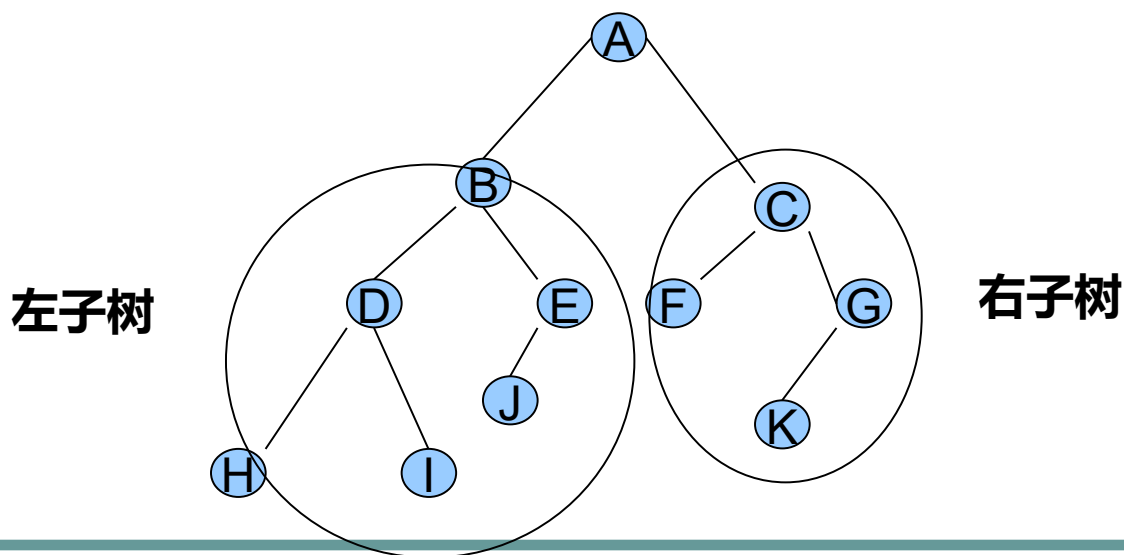
5.1.2 树的常用术语及运算

- 树的基本运算

- (7) 插入子树操作 $\text{addchild}(y,i,x)$ 。置以结点 x 为根的树为结点 y 的第 i 棵子树。若原树中无结点 y 或结点 y 的子树个数 $< i-1$ ，则空操作。
- (8) 删除子树操作 $\text{delchild}(x,i)$ 。删除结点 x 的第 i 棵子树。若无结点 x 或结点 x 的子树个数 $< i$ ，则空操作。
- (9) 遍历树 $\text{traverse}(T)$ 。按某个次序依次访问树中各个结点，并使每个结点只被访问一次。

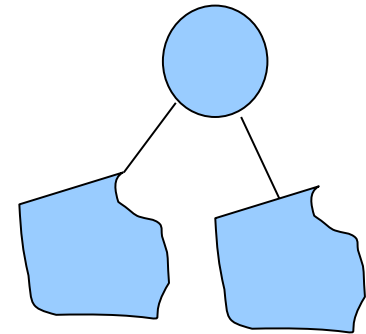
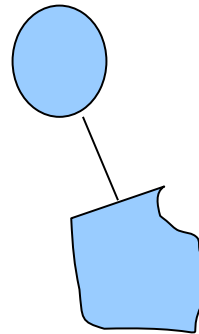
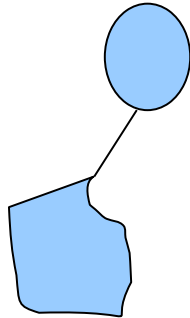
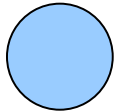
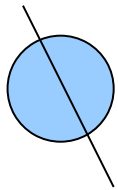
5.2 二叉树

- 5.2.1 二叉树的概念
- 每个结点至多只有二棵子树（即二叉树中不存在度大于 2 的结点），且二叉树的子树有左右之分，其次序不能任意颠倒



5.2.1 二叉树的概念

- 二叉树几种基本形态：



(a) 空二叉树 (b) 仅有根结点的二叉树

(c) 只有左子树的二叉树 (d) 只有右子树的二叉树 (e) 左、右子树均非空的二叉树

5.2.1 二叉树的概念

- 二叉树的基本操作：

- (1) 求根结点 $\text{ROOT}(\text{BT})$ 或 $\text{ROOT}(x)$ 。求二叉树 BT 的根结点或求结点 x 所在二叉树的根结点。若 BT 是空树或 x 不在任何二叉树上，则函数值为“空”。
- (2) 求双亲结点 $\text{PARENT}(\text{BT}, x)$ 。求二叉树 BT 中结点 x 的双亲结点。若结点 x 是二叉树 BT 的根结点或二叉树 BT 中无 x 结点，则函数值为“空”。
- (3) 求孩子（左孩子、右孩子）结点 $\text{LCHILD}(\text{BT}, x)$ 和 $\text{RCHILD}(\text{BT}, x)$ 。分别求二叉树 BT 中结点 x 的左孩子和右孩子结点。若结点 x 为叶子结点或不在二叉树 BT 中，则函数值为“空”。
- (5) 初始化二叉树 $\text{setnull}(\text{BT})$ 。置 BT 为空树。
- (5) 建树二叉树 $\text{CRT_BT}(x, \text{LBT}, \text{RBT})$ 。生成一棵以结点 x 为根、二叉树 LBT 和 RBT 分别为左、右子树的二叉树。

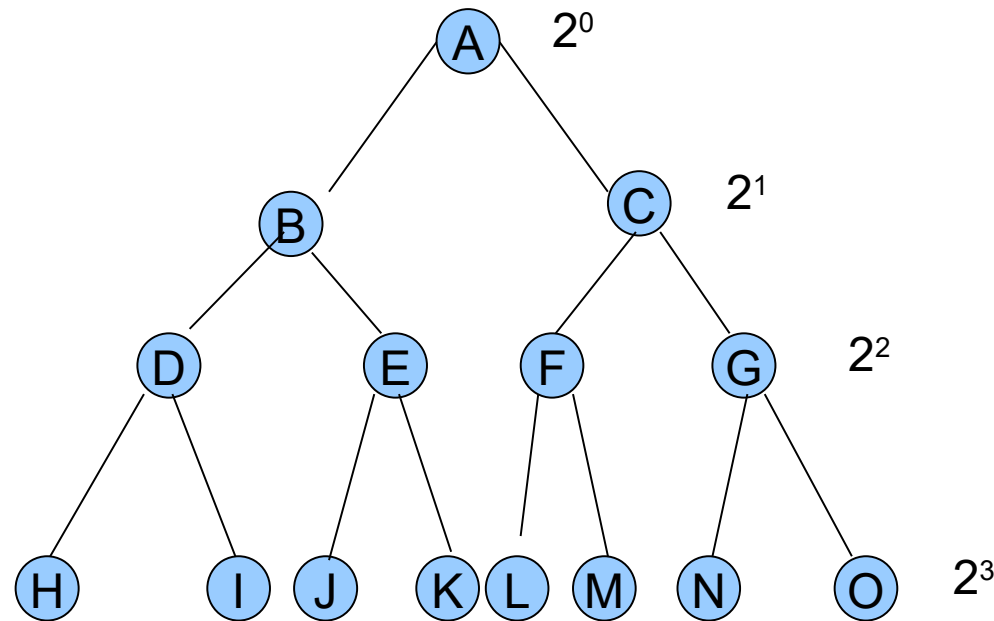
5.2.1 二叉树的概念

- 二叉树的基本操作：

- (6) 插入子树（左、右子树） $\text{INS_LCHILD}(\text{BT}, y, x)$ 和 $\text{INS_RCHILD}(\text{BT}, y, x)$ 。将以结点 x 为根且右子树为空的二叉树分别置为二叉树 BT 中结点 y 的左子树和右子树。若结点 y 有左子树 / 右子树，则插入后是结点 x 的右子树。
- (7) 删除子树（左、右子树） $\text{DEL_LCHILD}(\text{BT}, x)$ 和 $\text{DEL_RCHILD}(\text{BT}, x)$ 。分别删除二叉树中以结点 x 为根的左子树或右子树。若 x 无左子树或右子树，则空操作。
- (8) 遍历二叉树 $\text{TRAVERSE}(\text{BT})$ 。按某个次序依次访问二叉树中各个结点，并使每个 结点只被访问一次。

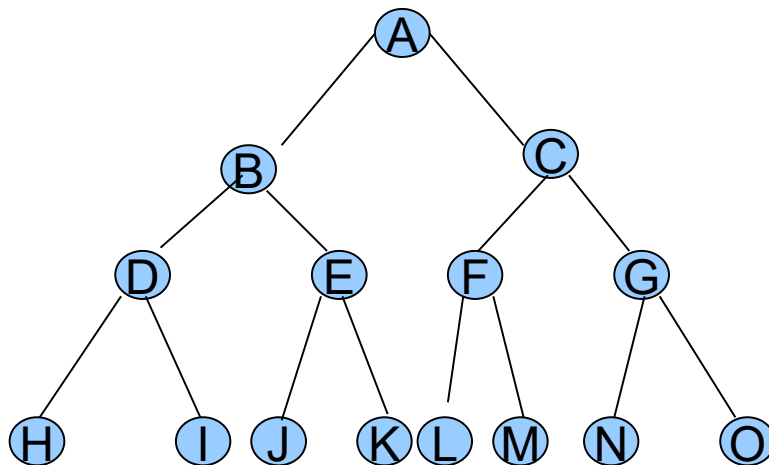
5.2.2 二叉树的性质

- **性质 1：** 在二叉树的第 i 层上至多有 2^{i-1} 个结点 ($i \geq 1$)。



5.2.2 二叉树的性质

- **性质 2**：深度为 k 的二叉树至多有 $2^k - 1$ 个结点 ($k \geq 1$), 最大结点数 $= 2^0 + 2^1 + \dots + 2^{k-1} = 2^k - 1$



5.2.2 二叉树的性质

- **性质 3**：对于任何一棵二叉树 T ，若其叶子结点数为 n_0 ，2 度结点数为 n_2 ，则

设二叉树 T 中 1 度结点数为 n_1 ，
则二叉树 T 结点总数为：

$$n_0 + n_1 + n_2 ;$$

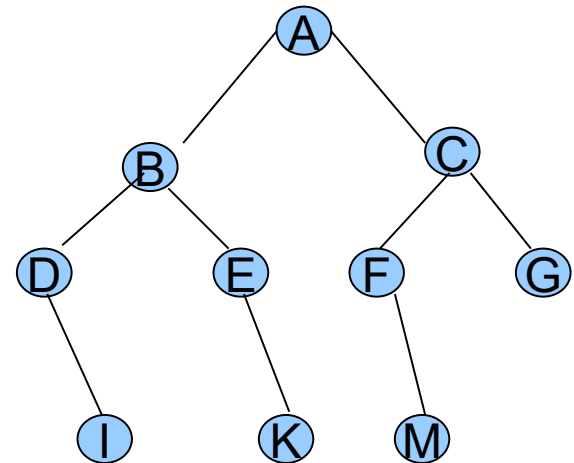
二叉树的分枝总数为：

$$n_1 + 2n_2 ,$$

显然有：

$$n_0 + n_1 + n_2 = n_1 + 2n_2 + 1$$

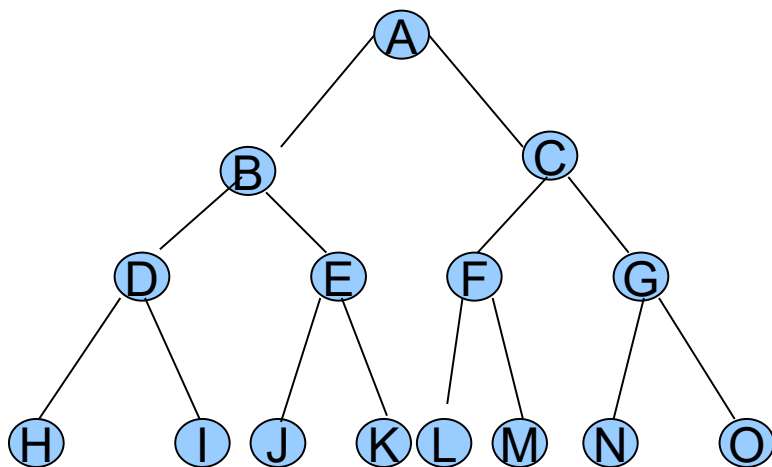
$$\therefore n_0 = n_2 + 1$$



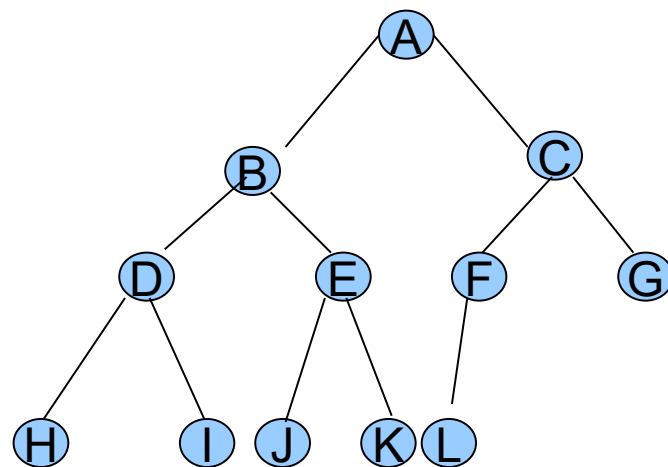
5.2.2 二叉树的性质

- 完全二叉树和满二叉树

一棵深度为 k 且有 $2^k - 1$ 个结点的二叉树称为满二叉树



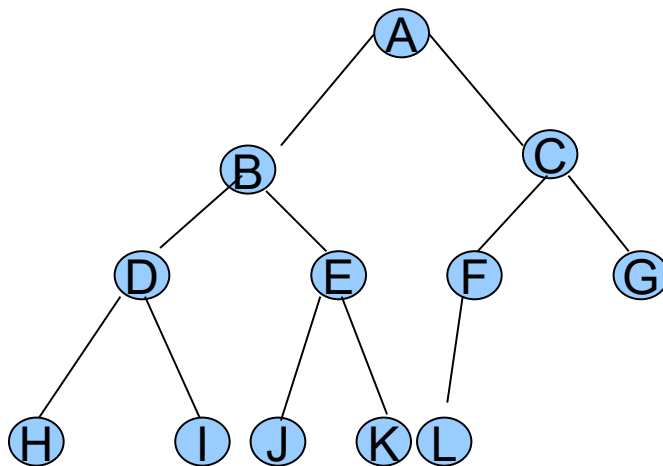
满二叉树



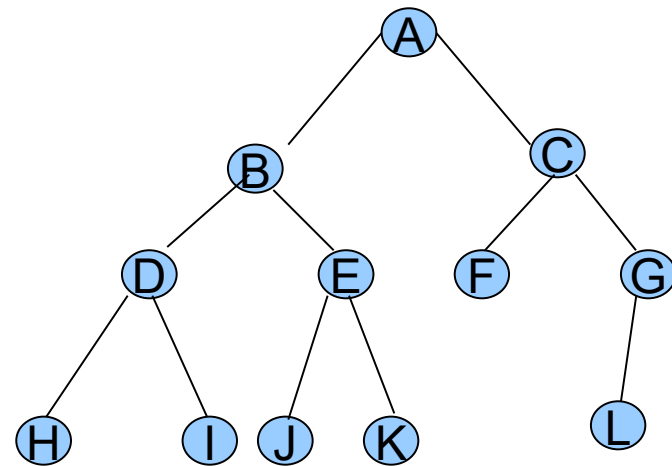
完全二叉树

二叉树的特殊形态—完全二叉树

- **完全二叉树**是指深度为 k 的、有 n 个结点的二叉树，当且仅当它的每一个结点都与深度为 k 的满二叉树中编号从 1 到 n 的结点一一对应。



完全二叉树



非完全二叉树

5.2.2 二叉树的性质

- **性质 5**：具有 n 个结点的完全二叉树的深度为不大于 $\log_2 n + 1$ 的整数

证明：假设深度为 k ，
则根据性质 2 和完全二叉树的定义有

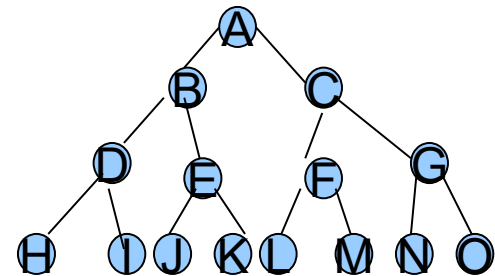
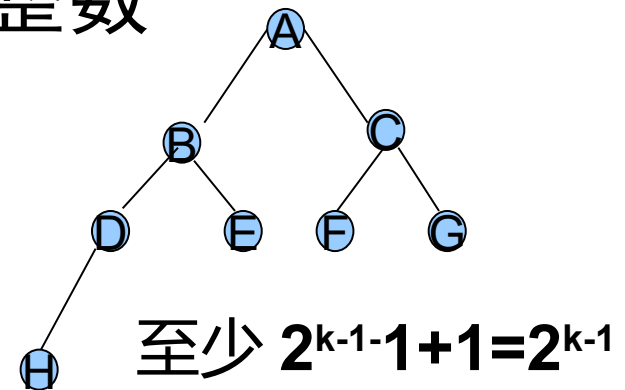
$$2^{k-1} - 1 + 1 \leq n \leq 2^k - 1$$

$$\text{即：} 2^{k-1} \leq n < 2^k$$

$$\text{于是 } k-1 \leq \log_2 n < k$$

$\because k$ 是整数

$$\therefore k = \lfloor \log_2 n \rfloor + 1$$



5.2.2 二叉树的性质

性质 5： 如果对一棵有 n 个结点的**完全二叉树**，则对任一结点 i ($1 \leq i \leq n$)，有：

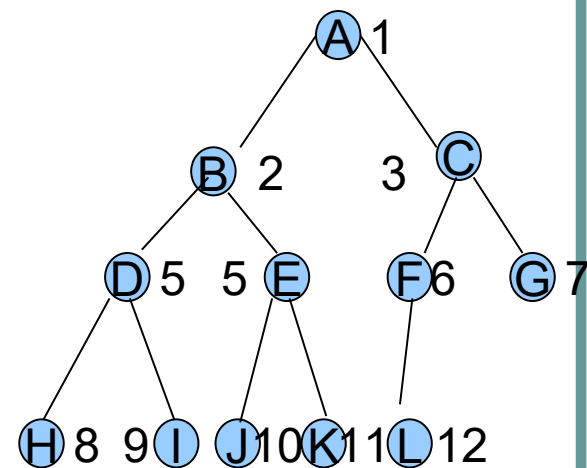
(1) 如果 $i=1$ ，则结点 i 是根结点，无双亲；如果 $i > 1$ ，则结点 i 的双亲结点编号为 $i/2$ 。

(2) 如果 $2i < n$ ，则结点 i 的左孩子编号为 $2i$ ；否则无左孩子。

(3) 如果 $2i+1 < n$ ，则结点 i 的右孩子编号为 $2i+1$ ；否则无右孩子。

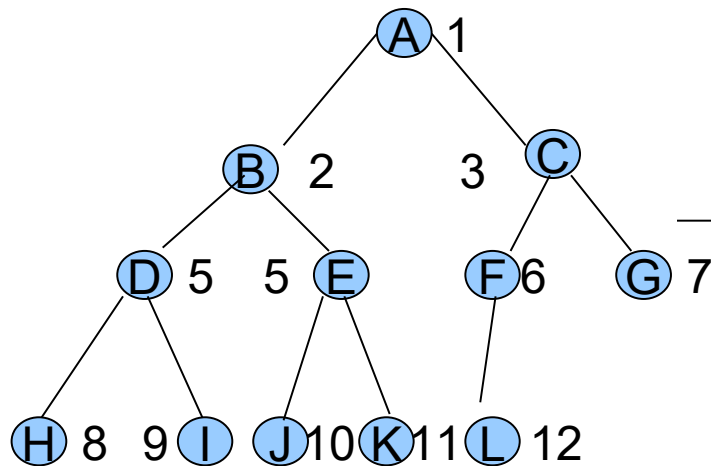
(5) 如果 i 为奇数且不为 1，则结点 i 的左兄弟编号为 $i-1$ ；否则无左兄弟。

(5) 如果 i 为偶数且小于 n ，则结点 i 的右兄弟编号为 $i+1$ ；否则无右兄弟。



5.2.3 二叉树的存储结构

(1) 顺序存储结构

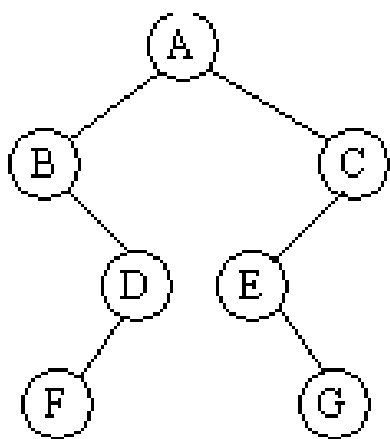


适合于完全二叉树

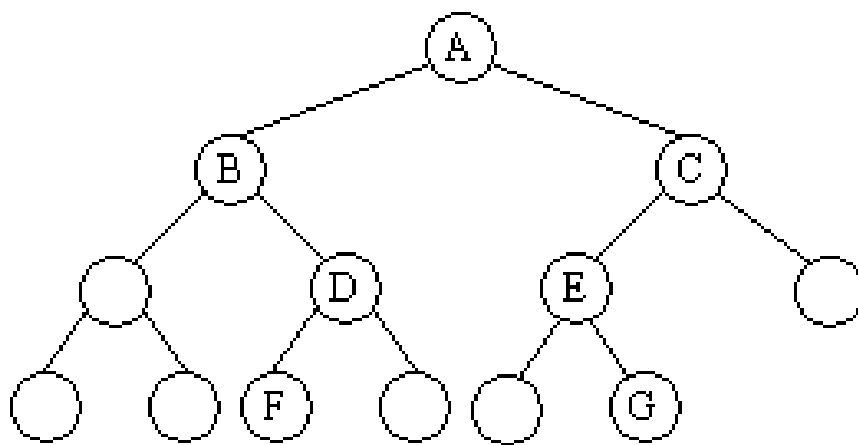
A
B
C
D
...
L

5.2.3 二叉树的存储结构

一般二叉树的顺序存储结构



(a) 二叉树



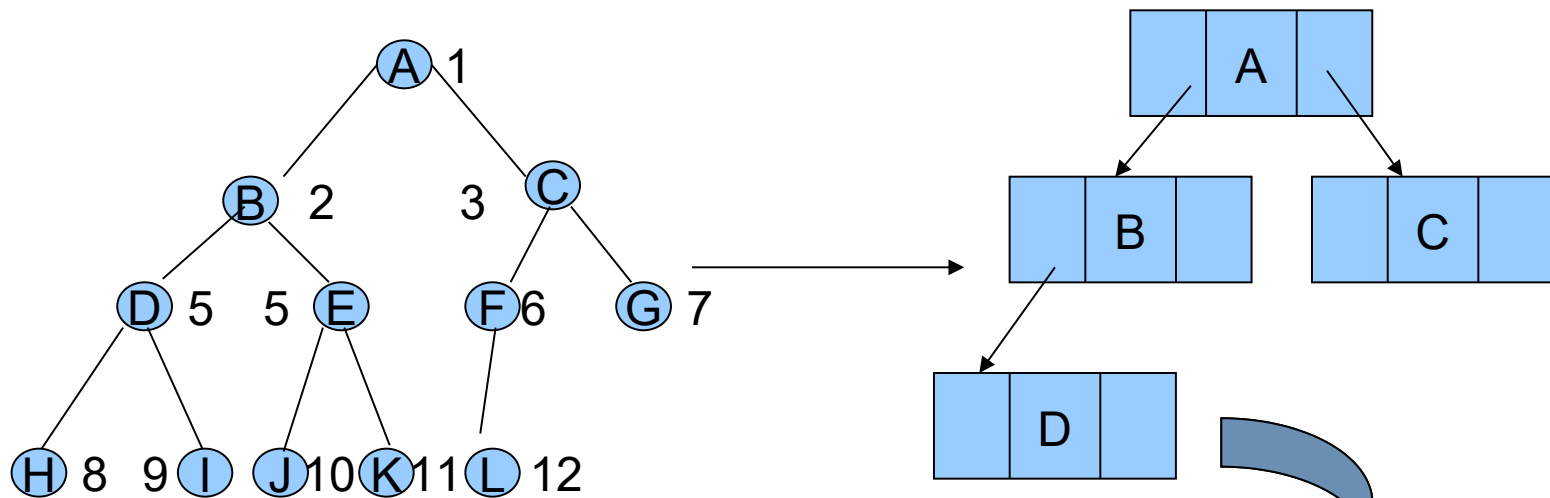
(b) 添加空结点的完全二叉树

A	B	C	^	D	E	^	^	^	F	^	^	G
---	---	---	---	---	---	---	---	---	---	---	---	---

(c) 顺序存储状态

5.2.3 二叉树的存储结构

(2) 链式存储结构



```
typedef struct btnode
{ elemtp data;
  struct node
  *lchild,*rchild;
}bitnode;
typedef bitnode *bitree
```

5.2.3 二叉树的存储结构

(2) 链式存储结构

如果经常需要求双亲可以设置一指向双亲的指针

左孩子	数据	右孩子	双亲
-----	----	-----	----

5.2.5 二叉树的简单运算实现

(1) 求根节点:

```
elemtp root(bitree bt)
{ if (bt==NULL) return NULL;
  return bt->data;
}
```

5.2.5 二叉树的简单运算实现

(2) 建立二叉树操作

```
bitree create(elemtype x,bitree lbt,bitree  
    rbt) /* 该运算生成一棵以 x 为根结点,  
        lbt 和 rbt 分别为左右子树的二叉树 */  
{bitree p;  
    p=(bitree)malloc(sizeof(bitnode));  
    p->data=x;  
    p->lchild=lbt;  
    p->rchild=rbt;  
    return p;                /* 返回建成的二叉树 */  
}
```


5.2.5 二叉树的简单运算实现

(3) 插入左孩子:

```
void add_lchild(bitree bt,elemtp x)
```

```
{ bitree p;
```

```
    p=(bitree)malloc(sizeof(bitnode));
```

```
    p->data=x;
```

```
    p->lchild=NULL;
```

```
    p->rchild=NULL;
```

```
    bt->lchild=p;    /* 插入 bt 的左孩子域 */
```

```
}
```

5.2.5 二叉树的简单运算实现

(5) 删除左孩子:

```
void delete_lchild(bitree bt)
{
    bitree p;
    p=bt->lchild; /* 保存左子树指针 */
    bt->lchild=NULL;
    free (p);      /* 释放左子树空间 */
}
```

5.3 二叉树的遍历

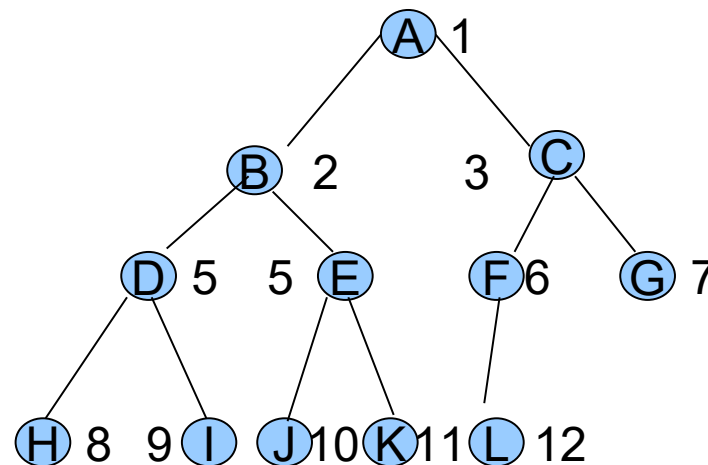
- 二叉树遍历：即如何按某条搜索路径巡访树中每个结点，使得每个结点均被访问一次。而且仅被访问一次。
- 主要有以下几种：
 - 先序遍历
 - 中序遍历
 - 后序遍历
 - 层次遍历

5.3 二叉树的遍历

● 先序遍历

若二叉树为空，则空操作，否则，

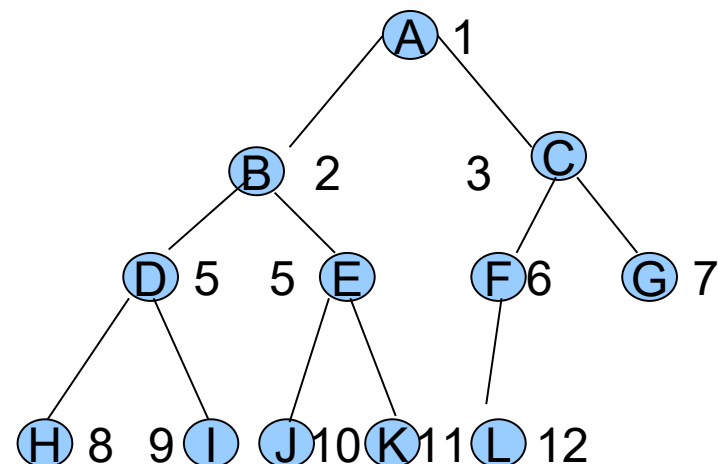
- (1) 访问根结点；
- (2) 先序遍历左子树；
- (3) 先序遍历右子树



A	B	D	H	I	E	J	K	C	F	L	G
---	---	---	---	---	---	---	---	---	---	---	---

5.3 二叉树的遍历

- 中序遍历
- 若二叉树为空，则空操作；否则，
 - (1) 中序遍历左子树；
 - (2) 访问根结点，
 - (3) 中序遍历右子树。



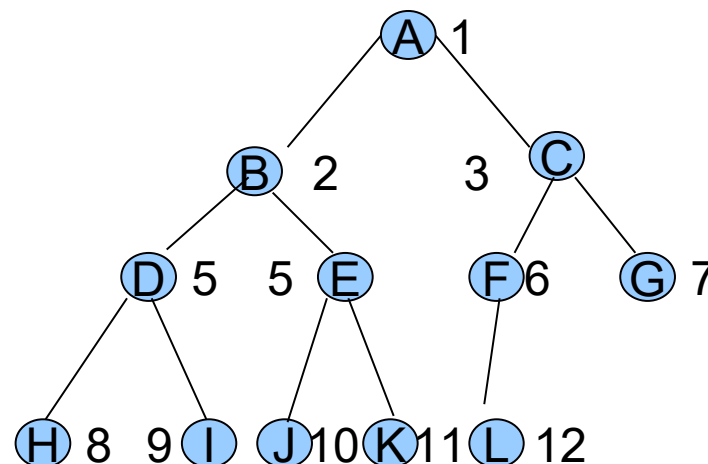
H	D	I	B	J	E	K	A	L	F	C	G
---	---	---	---	---	---	---	---	---	---	---	---

5.3 二叉树的遍历

● 后序遍历

若二叉树为空，则
空操作；否则，

- (1) 后序遍历左子树；
- (2) 后序遍历右子树；
- (3) 访问根结点。



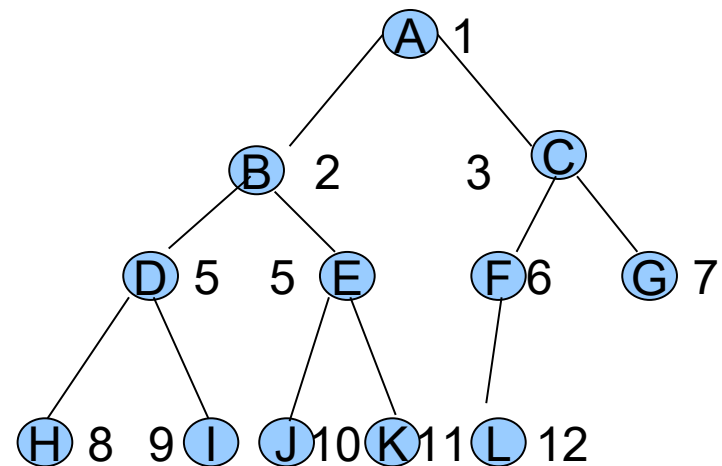
H	I	D	J	K	E	B	L	F	G	C	A
---	---	---	---	---	---	---	---	---	---	---	---

5.3 二叉树的遍历

- 层次遍历:

若二叉树为空，
则空操作；

否则，从根结点
开始，自顶向下
访问各层，从左
到右访问同一层
的结点。

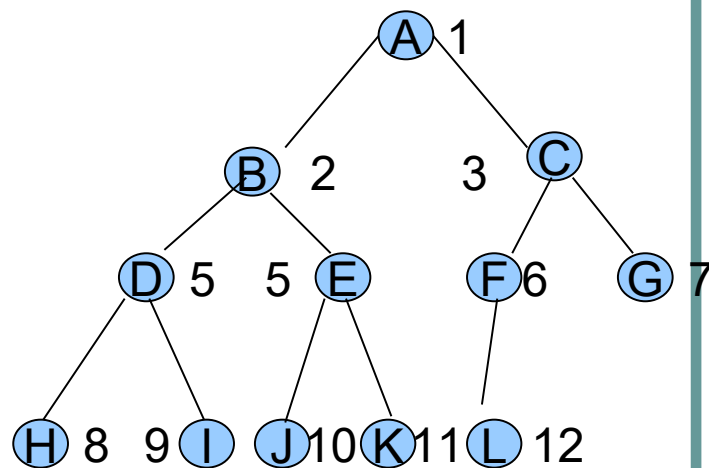


A B C D E F G H I J K L

5.3.1 遍历二叉树的递归算法

(1) 先序遍历二叉链表的递归算法：

```
void preorder(bitree bt)
{ if(bt==NULL)
  return ; /* 递归结束条件 */
  else
    {printf("%d\t",bt->data);
      /* 前序遍历左子树 */
      preorder(bt->lchild);
      /* 前序遍历右子树 */
      preorder(bt->rchild);
    }
}
```

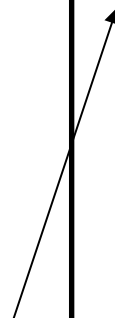


5.3.1 遍历二叉树的递归算法

(1) 先序遍历二叉链表的递归算法：

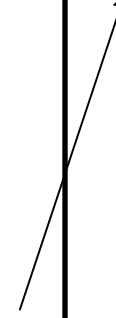
```
preorder(A)
{ if(A==NULL)
  return ;
else
  { printf("A");

    preorder(B);
    preorder(C);
  }
}
```



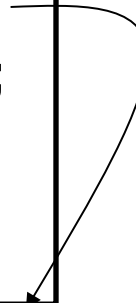
```
preorder(B)
{ if(B==NULL)
  return ;
else
  { printf("B");

    preorder(D);
    preorder(E);
  }
}
```



```
preorder(D)
{ if(B==NULL)
  return ;
else
  { printf("D");

    preorder(H);
    preorder(I);
  }
}
```

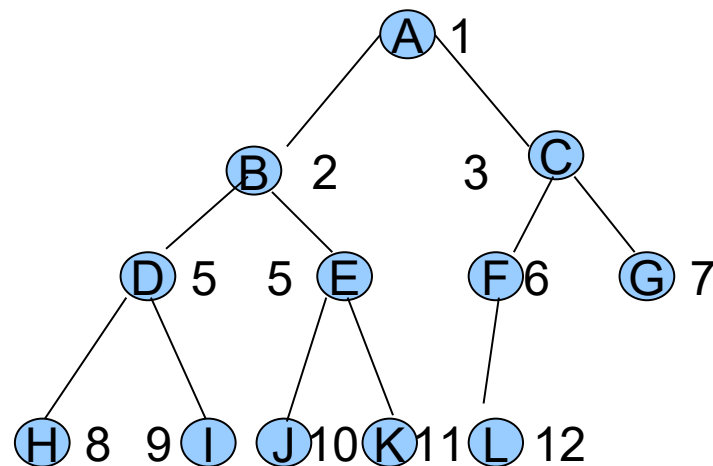


```
preorder(H)
```

5.3.1 遍历二叉树的递归算法

(2) 中序遍历二叉链表的递归算法:

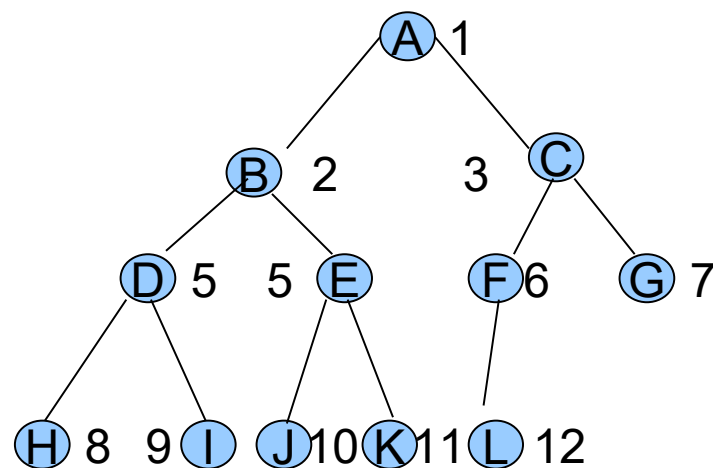
```
void inorder(bitree bt)
{if(bt==NULL)
    return ; /* 递归结束条件 */
else
    { /* 中序遍历左子树 */
      inorder(bt->lchild);
      /* 访问根结点 */
      printf("%d\t",bt->data);
      /* 中序遍历右子树 */
      inorder(bt->rchild);
    }
}
```



5.3.1 遍历二叉树的递归算法

(3) 后序遍历二叉链表的递归算法:

```
void postorder(bitree bt)
{
    if(bt==NULL)
        return ; /* 递归结束条件 */
    else
    {
        postorder(bt->lchild);
        /* 后序遍历左子树 */
        postorder(bt->rchild);
        /* 后序遍历右子树 */
        printf("%d\t",bt->data);
        /* 访问根结点 */
    }
}
```



5.3.2 遍历二叉树的非递归算法

- 使用栈或队列，可以实现二叉树遍历的非递归算法。
- (1) 先序遍历二叉链表的非递归算法：

```
#define MAXSIZE 100
```

```
void nrpreorder(bitree bt)
```

```
{bitree stack[MAXSIZE],p;
```

```
int top=0;
```

```
p=bt;
```

```
do
```

```
{while(p!=NULL)
```

```
{printf("%d\t",p->data);
```

```
if(p->rchild!=NULL) /* 如果右子树不空 */
```

```
stack[++top]=p->rchild; /* 右孩子进栈 */
```

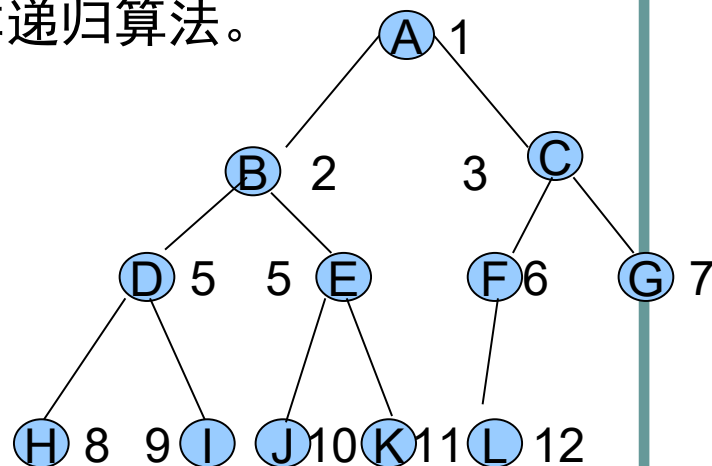
```
p=p->lchild;}
```

```
if(top>0)
```

```
p=stack[top--];
```

```
}while(top>0); /* 当栈不空时继续遍历 */
```

```
}
```



思路：沿左子树一直深入，并把所遇到结点的非空右孩子进栈；访问完左孩子后，将右孩子出栈遍历其右子树。

5.3.2 遍历二叉树的非递归算法

- (2) 中序遍历二叉链表的非递归算法:

```
#define MAXSIZE 100
```

```
void nrinorder(bitree bt)
```

```
{bitree stack[MAXSIZE],p;
```

```
int top=0;
```

```
p=bt;
```

```
do
```

```
{while(p!=NULL)
```

```
{ stack[++top]=p; /* 所遇结点进栈 */
```

```
p=p->lchild; /* 继续搜索 p 的左子树 */
```

```
if(top>0)
```

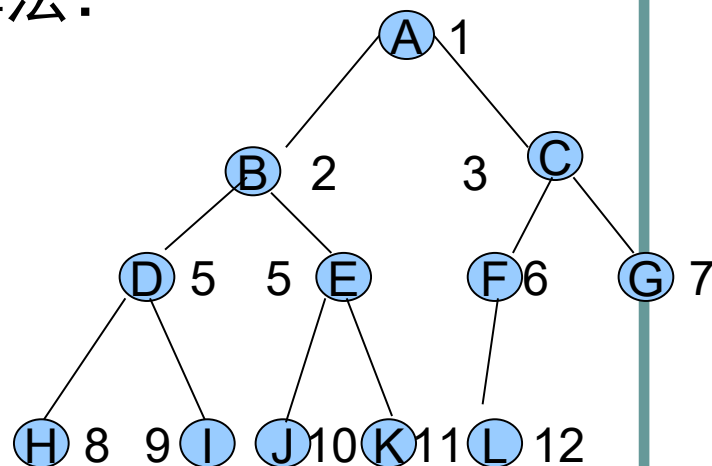
```
{p=stack[top--]; /* 出栈一个结点 */
```

```
printf("%d\t",p->data); /* 访问结点 */
```

```
p=p->rchild; /* 继续搜索右子树 */
```

```
}while(top>0);
```

```
}
```

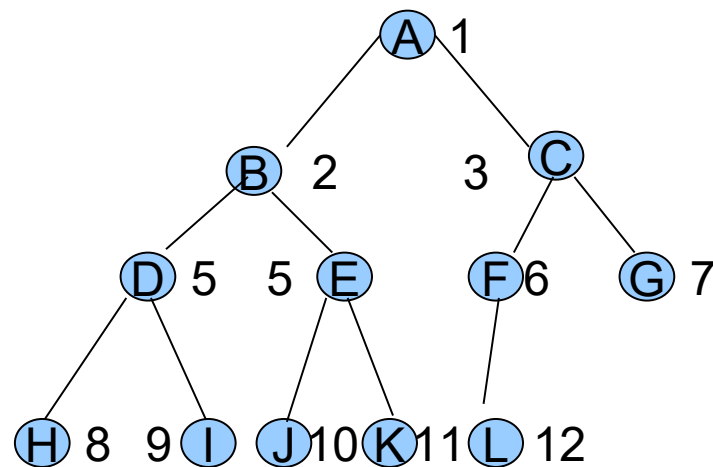


思路：与前序遍历类同，只是沿左子树向下搜索的过程中先将所遇结点进栈，待遍历完左子树返回时从栈顶退出结点并访问，然后再遍历右子树。

5.3.2 遍历二叉树的非递归算法

(3) 二叉树的层次遍历:

```
void layer_travel_bitree(bitree *bt)
{ bitree *p;
  // 初始化队列 Q，队列的元素
  // 为指向 bitree 结点的指针类型
  init_Queue(Q);
  // 根结点地址入队
  in_queue(Q, bt);
  while(!queue_empty(Q))
  { p=out_queue(Q); // 出队
    visit(p);
    // 左孩子结点地址入队
    if(p->lp!=NULL) in_queue(Q, p->lp);
    // 右孩子结点地址入队
    if(p->rp!=NULL) in_queue(Q, p->rp); }
}
```



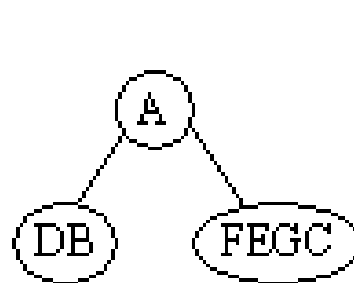
5.3.3 遍历序列与二叉树的复原

- 必需至少提供 2 个不同的遍历序列，才能复原唯一的二叉树。

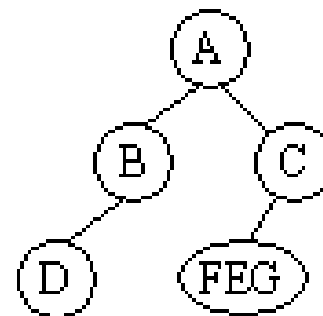
1. 利用前序序列和中序序列恢复二叉树

例：前序 ABDCEFG

中序 DBAFEGC



(a) 第一层划分



(b) 第二层划分

思路：前序序列确定根结点

中序序列确定左子树和右子树

5.3.3 遍历序列与二叉树的复原

2. 利用后序序列和中序序列恢复二叉树

例：后序 DBFGECA

中序 DBAFEGC

思路：后序序列确定根结点

中序序列确定左子树和右子树

5.3.3 遍历序列与二叉树的复原

- 必需至少提供 2 个不同的遍历序列，才能复原唯一的二叉树。

1. 利用前序序列和中序序列恢复二叉树

例：前序 ABDCEFG

中序 DBAFEGC

5.3.5 基于遍历的几种二叉树运算的实现和应用举例

1. 查找结点 x 的运算
2. 求二叉树中的叶子结点数目
3. 求二叉树的高度

1. 查找结点 x 的运算

- 查找二叉树 bt 中的结点 x ，可以结合在四种遍历算法中的任何一个算法中进行。在此我们以前序遍历来实现查找运算的递归算法。

```
bitree search(bitree bt, elemtype x)
{ if(bt!=NULL)
  {if(bt->data==x)
    return bt;
   search(bt->lchild, x);    /* 在左子树中查找 */
   search(bt->rchild, x);    /* 在右子树中查找 */
  }
  else return NULL;
}
```

2. 求二叉树中的叶子结点数目

- 在遍历过程中对所遇结点判断是否为叶子结点，若是则统计变量加 1，直到遍历完所有结点，叶子结点总数即可求得。下面给出利用中序遍历求叶子结点数的递归算法：

```
int countleaf(bitree bt)
{
    int num=0;
    if(bt!=NULL)
        if((bt->lchild==NULL)&&(bt->rchild==NULL))
            num++;
        else
            num=countleaf(bt->lchild)+countleaf(bt->rchild);
    return num;
}
```

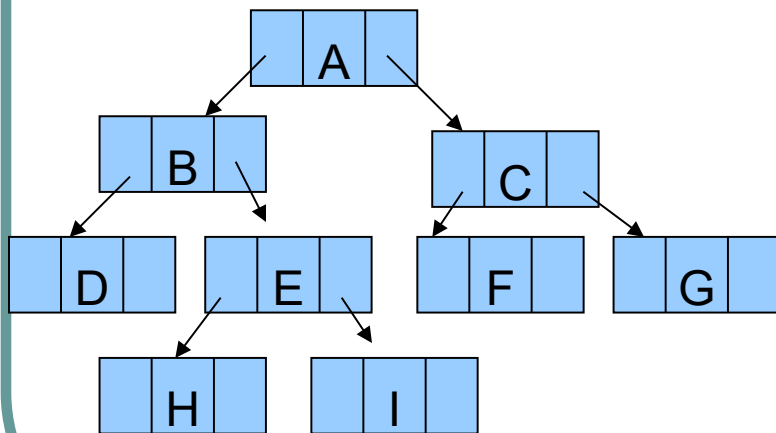
3. 求二叉树的高度

- 可以在二叉树的前序或后序遍历过程中求出，下面给出的算法是在后序遍历过程中求深度的递归算法。

```
int treedepth(bitree bt)
{
    int h, lh, rh;
    if (bt == NULL)    h = 0;
    else
    {
        lh = treedepth(bt->lchild);    /* 左子树高度赋 lh */
        rh = treedepth(bt->rchild);    /* 右子树高度赋 rh */
        if (lh > rh)
            h = lh + 1;
        else
            h = rh + 1;
    }
    return h;
}
```

5.5 线索二叉树

- 5.5.1 线索二叉树的概念
- 思考：二叉树的遍历产生了一线性序列，如何不再通过重新遍历二叉树，而能够直接找到任一结点的前驱和后继呢？



方法一： 结点中增加指向
前驱和后继的指针

左孩子	前驱	数据	右孩子	后继

缺点： 空间利用率低

5.5.1 线索二叉树的概念

方法二：增加两个标志位 rtag 和 ltag, 利用现有的空指针域，n 个结点的二叉树必有 n+1 个空指针域

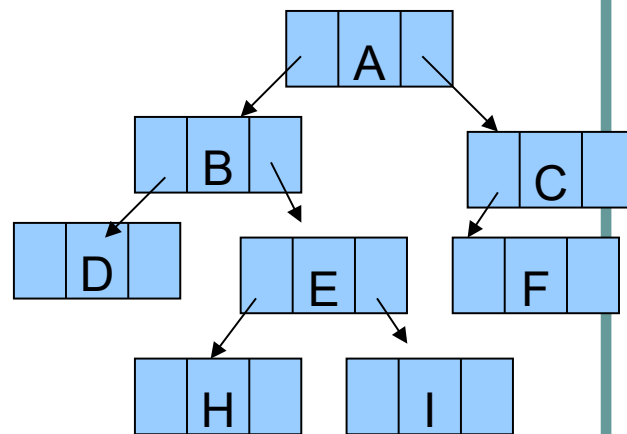
左孩子	ltag	数据	右孩子	rtag
-----	------	----	-----	------

$$rtag = \begin{cases} 0 \\ 1 \end{cases}$$

表示 rchild 域为指向右孩子的指针；
表示 rchild 域为指向其后继的右线索。

$$ltag = \begin{cases} 0 \\ 1 \end{cases}$$

表示 lchild 域为指向左孩子的指针；
表示 lchild 域为指向其前趋的左线索。



线索二叉树的类型定义

- `typedef enum{0, 1} ptrtag;`
- `typedef struct bithnode`
- `{elemtype data;`
- `struct bithnode *lchild,*rchild;`
- `ptrtag ltag,rtag;`
- `}bithrnode;`
- `typedef bithrnode *bithrtree;`

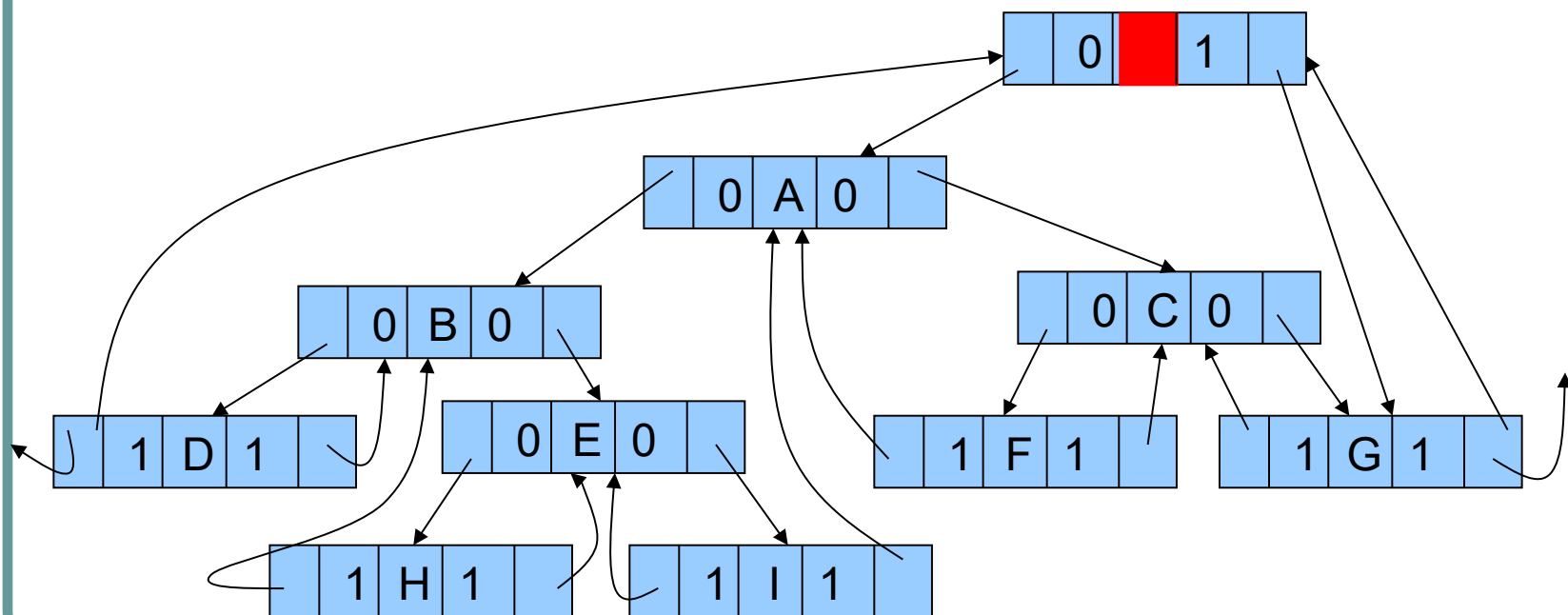
$$rtag = \begin{cases} 0 & \text{表示 rchild 域为指向右孩子的指针;} \\ 1 & \text{表示 rchild 域为指向其后继的右线索。} \end{cases}$$

$$ltag = \begin{cases} 0 & \text{表示 lchild 域为指向左孩子的指针;} \\ 1 & \text{表示 lchild 域为指向其前趋的左线索。} \end{cases}$$

5.5.1 线索二叉树的概念

中序线索二叉树举例

中序序列 DBHEIAFCG



二叉树中序线索后的结果

5.5.2 线索二叉树的构造算法

- 建立线索二叉树的过程称作对二叉树**线索化**，线索化需要在遍历的过程中来实现。
- 在对二叉树的某种次序遍历过程中，一边遍历一边建立线索，
 - 若当前访问结点的左孩子域为空则建立**前趋线索**，
 - 若右孩子域为空则建立**后继线索**。
- 为实现这一过程，可设指针 pre 指向刚刚访问过的结点，指针 p 指向当前结点，就可以方便前趋和后继线索的填入。

5.5.2 线索二叉树的构造算法

中序遍历线索化二叉链表的算法：

其中：函数inorderthr处理头结点以及与头结点有关的指针和线索，包括对于空二叉树的特殊处理；并调用函数inthreading对非空二叉树进行中序线索化。
算法的描述如下：

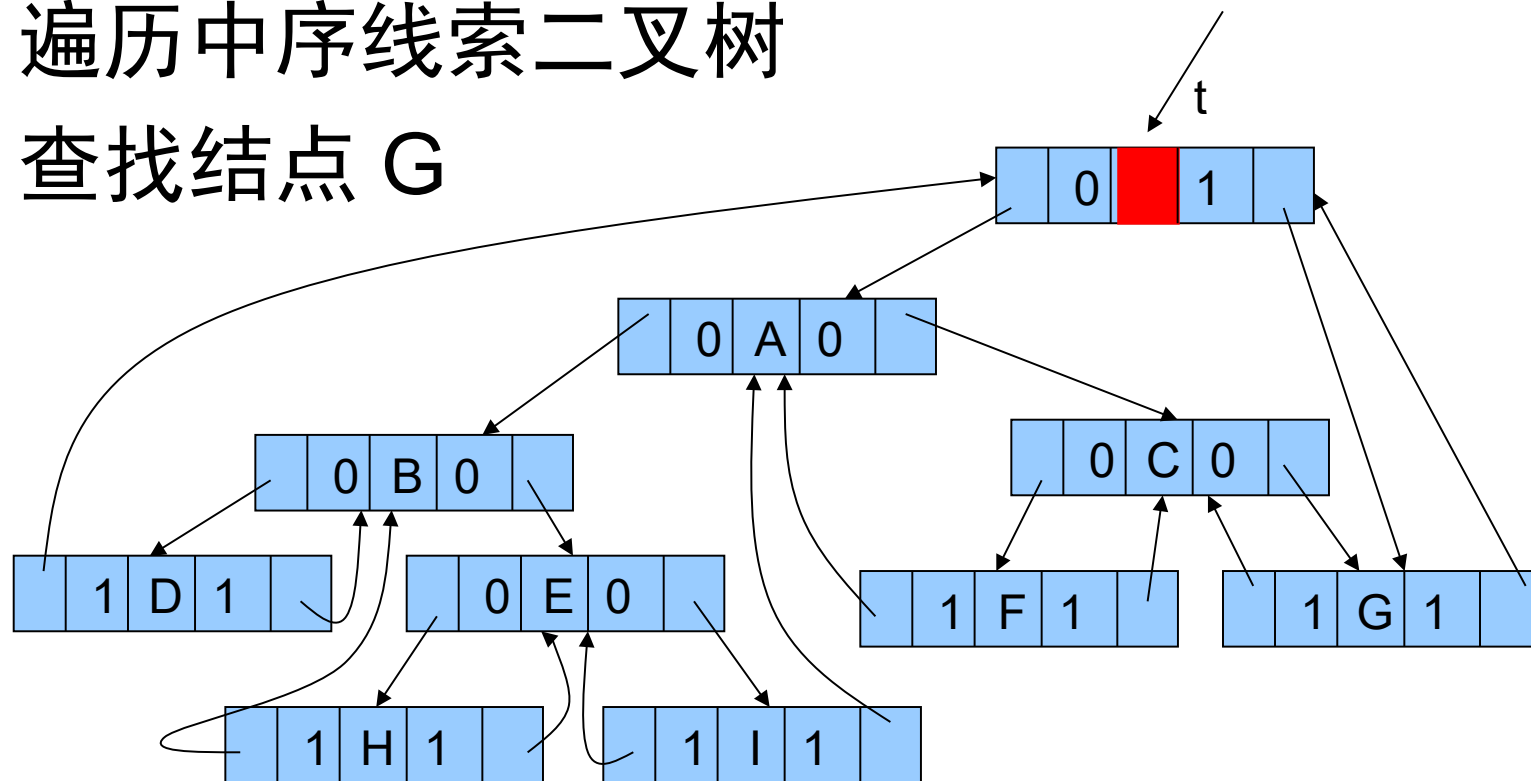
```
bithrtree inorderthr(bithrtree t)
{
    bithrtree thrt;
    thrt=(bithrtree)malloc(sizeof(bithrnode));
    thrt->ltag=0;
    thrt->rtag=1;
    if(t==NULL)
    {
        thrt->lchild=thrt;
        thrt->rchild=thrt;
    }
    else
    {
        thrt->lchild=t;
        pre=thrt;
        inthreading(t);
        pre->rchild=thrt;
        pre->rtag=1;
        thrt->rchild=pre;
    }
    return thrt;
}

void inthreading(bithrtree t)
    /*在中序遍历过程中线索化二叉树*/
```

5.5.3 线索二叉树上的运算实现

遍历中序线索二叉树

查找结点 G



思路：先找到最左结点，然后不断找后继，直到回到头结点；查结点后继时若遇到右孩子不为空时，后继即为右子树的最左下结点。

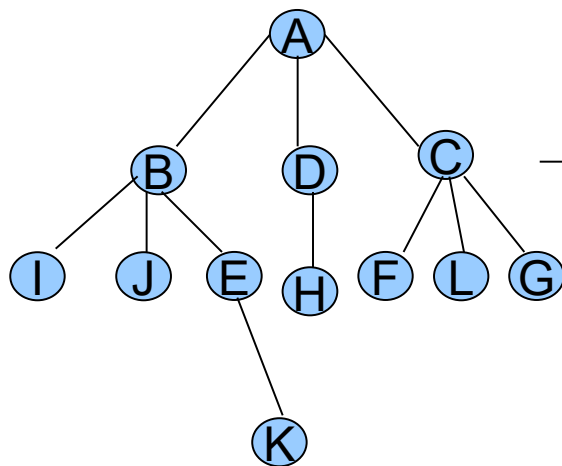
5.5.3 线索二叉树上的运算实现

例：中序线索二叉树的中序遍历：

```
void inorderthr(bithrtree t)
{ bithrtree p;
  p=t->lchild;
  while(p!=t)          /* 空树或遍历结束时 p=t */
  { while(p->ltag==0)
    { p=p->lchild;      /* 找最左下结点 */
      printf( "%d\t", p->data);
      while((p->rtag==1)&&(p->rchild!=t))
        {p=p->rchild;
          printf( "%d\t", p->data);}
        p=p->rchild; }
    }
```

5.5 树和森林

- 5.5.1 树和森林的存储结构
- 1、双亲表示法



0	A	-1
1	B	0
2	D	0
3	C	0
5	I	1
5	J	1
6	...	
7	G	3
8	K	6

5.5.1 树和森林的存储结构

1、双亲表示法

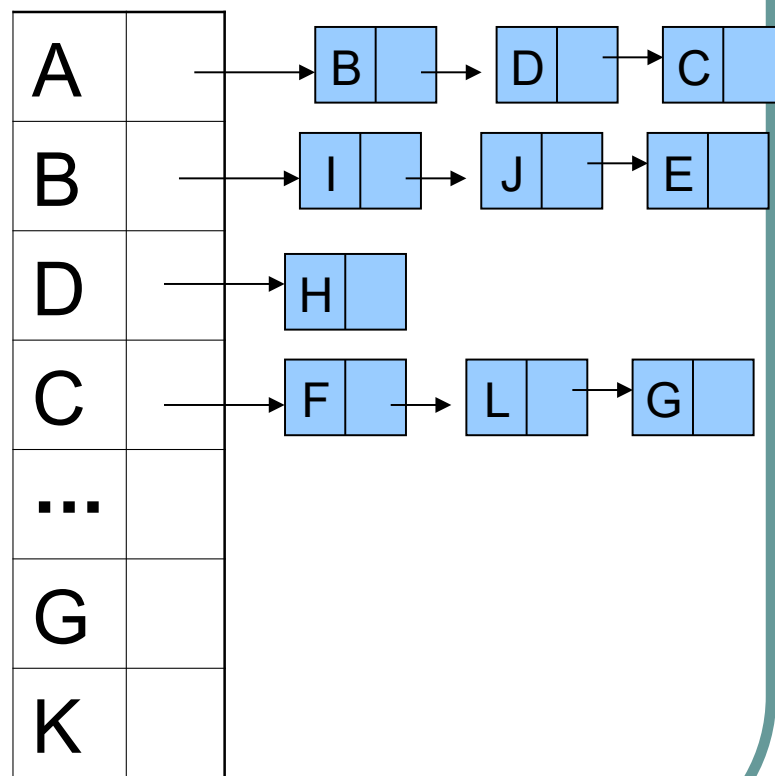
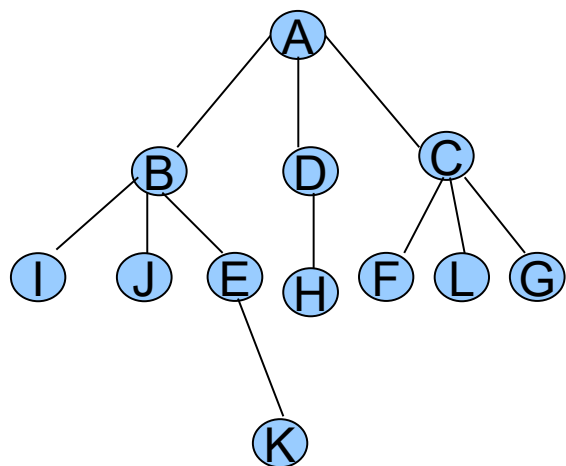
- 结点定义：

```
typedef struct node
{
    elemtp data;
    int parent; // 指示双亲结点的下标
}
typedef struct
{
    node tree[maxlen];
    int num; // 结点个数
}tree_parent;
```

0	A	-1
1	B	0
2	D	0
3	C	0
5	I	1
5	J	1
6	...	
7	G	3
8	K	6

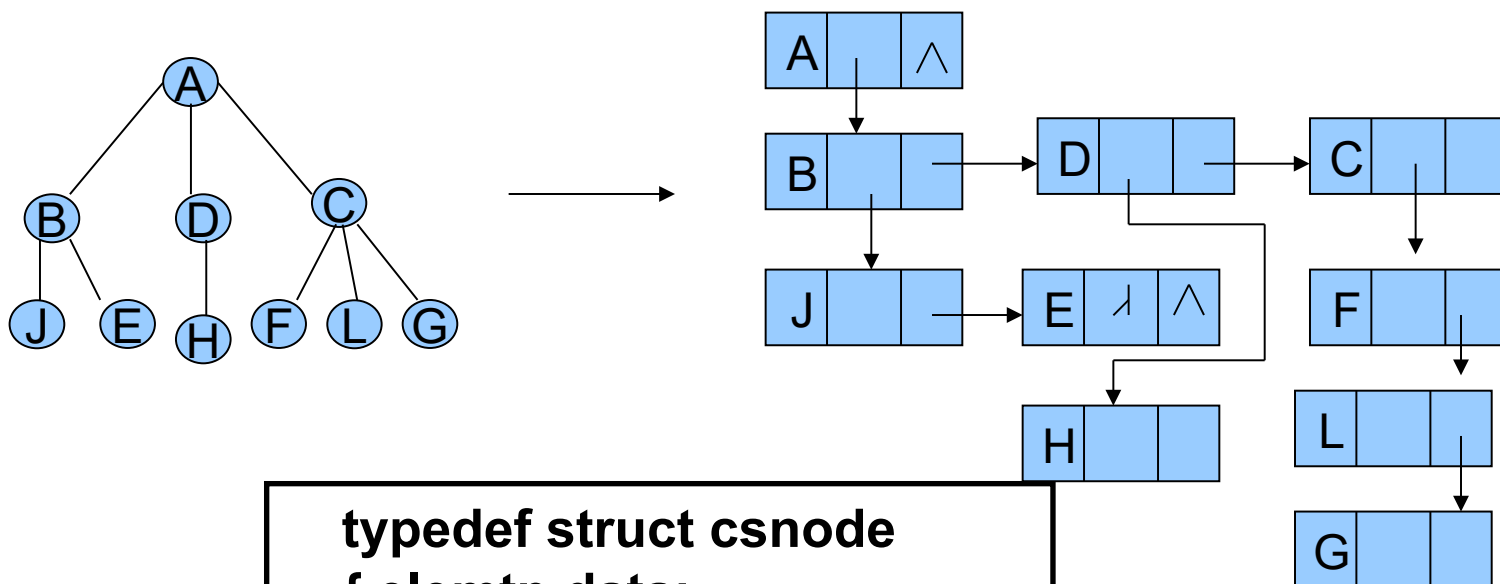
5.5.1 树和森林的存储结构

● 2、孩子表示法



5.5.1 树和森林的存储结构

● 3、孩子兄弟表示法

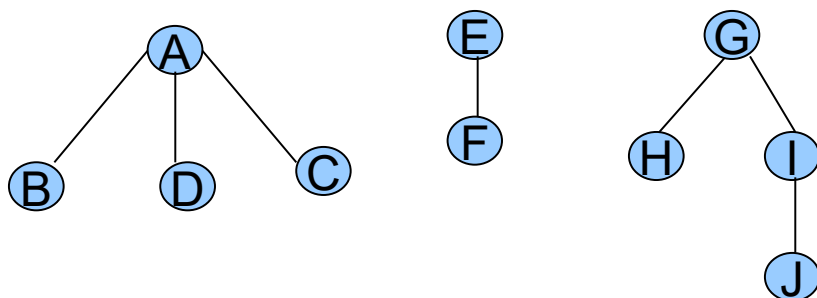


```
typedef struct csnode
{ elemtp data;
  struct csnode *first,*next;
}cstree;
```

5.5.2 树和森林与二叉树之间的转换

1. 森林转换成二叉树

例



方法:

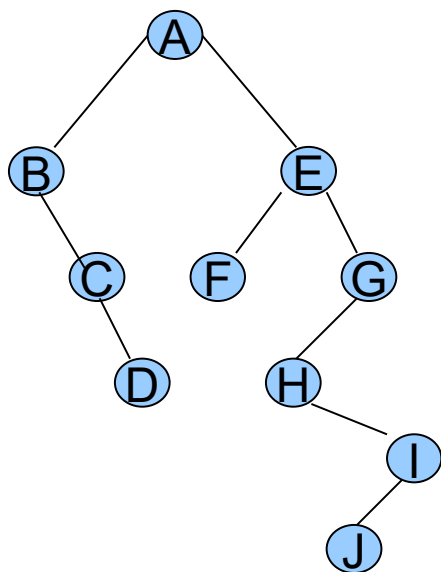
依次将每棵树都转换成二叉树; 最后将每棵树作为前一棵二叉树的右子树;

每棵树转换法则: 将根结点第一个孩子转为其左孩子, 其他孩子转换为第一个孩子的右边孩子;

5.5.2 树和森林与二叉树之间的转换

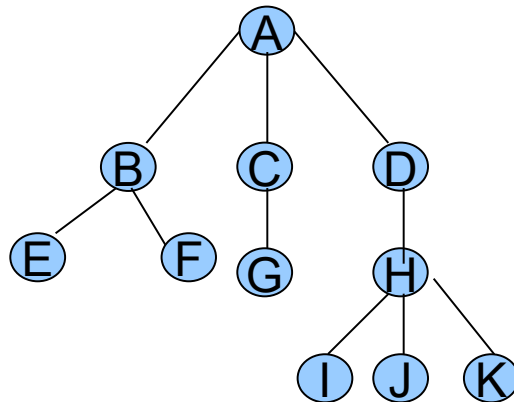
2. 二叉树转换成森林

逆过程，主要时将右子树断开，
断开的原则是其左孩子不为空



5.5.3 树和森林的遍历

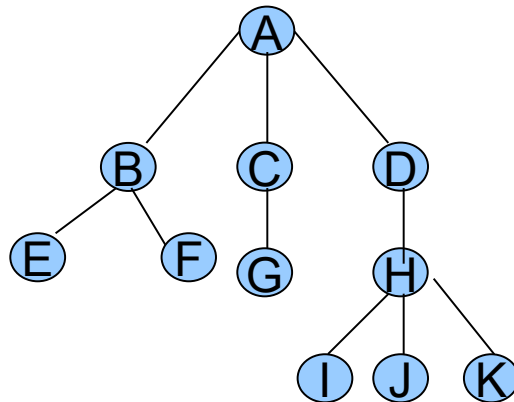
- 由树的结构定义可以引出树的两种遍历：
- 先（根）序遍历：先访问根结点，然后先序遍历每棵子树；对应于二叉树的先根序遍历。



先根序遍历序列是： A,B, E,F,C,G,D,H,I,J,K

5.5.3 树和森林的遍历

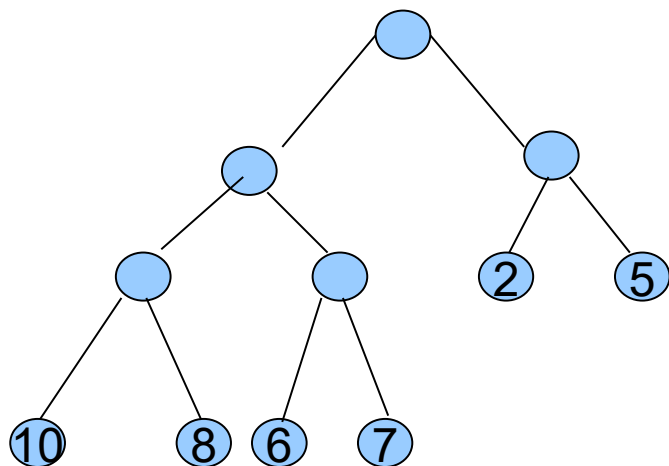
- 后（根）序遍历：先依此后序遍历每棵子树，然后访问根结点。对应二叉树的中根序遍历。



先根序遍历序列是： E,F,B,G,C,I,J,K,H,D,A

5.6 哈夫曼树

● 5.6.1 哈夫曼树的概念及其构造算法



带权树

1. 路径和路径长度

2. 结点的权和带权
路径长度

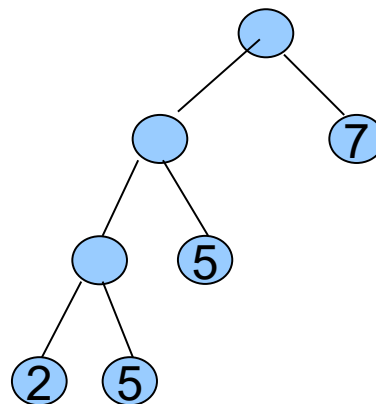
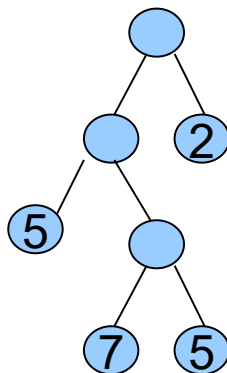
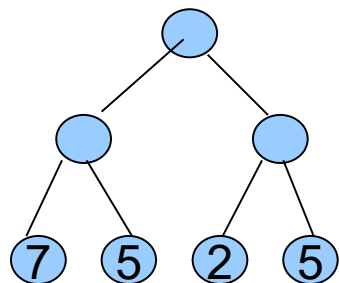
3. 树的带权路径长
度

$$\sum_{i=1}^N W_i L_i$$

WPL =

5.6.1 哈夫曼树的概念及其构造算法

- 5. 哈夫曼树：最优二叉树，带权路径长度最小的树
- 例：叶子结点的权为 7,5,2,5



5.6.1 哈夫曼树的概念及其构造算法

● 5. 哈夫曼算法

Huffman 给出了一个建立哈夫曼树的算法：

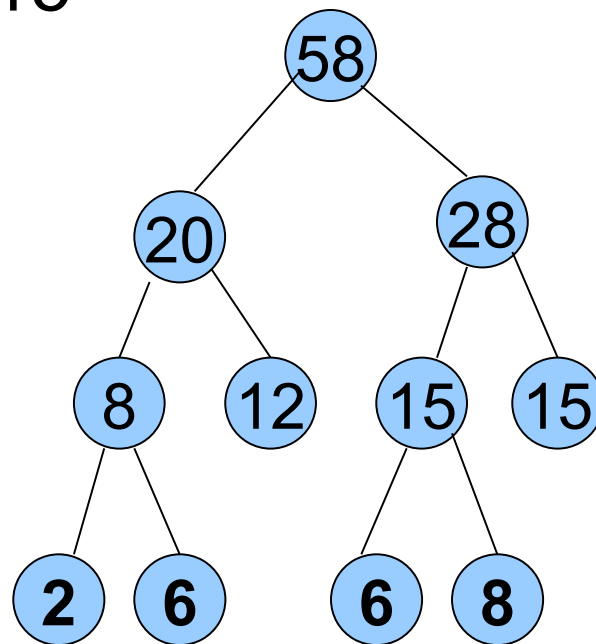
例： 2， 6， 6， 8， 12， 15

(1) 根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 构成 n 棵二叉树的集合 $F = \{T_1, T_2, \dots, T_n\}$ ，其中每棵二叉树 T_i 中只有一个带权为 w_i 的根结点，其左右子树均空。

(2) 在 F 中选取两棵根结点的权值最小的树作为左右子树构造一棵新的二叉树，且置新的二叉树的根结点的权值为其左、右子树上根结点的权值之和。

(3) 在 F 中删除这两棵树，同时将新得到的二叉树加入 F 中。

(4) 重复(2)和(3)，直到 F 只含一棵树为止，这棵树便是哈夫曼树。

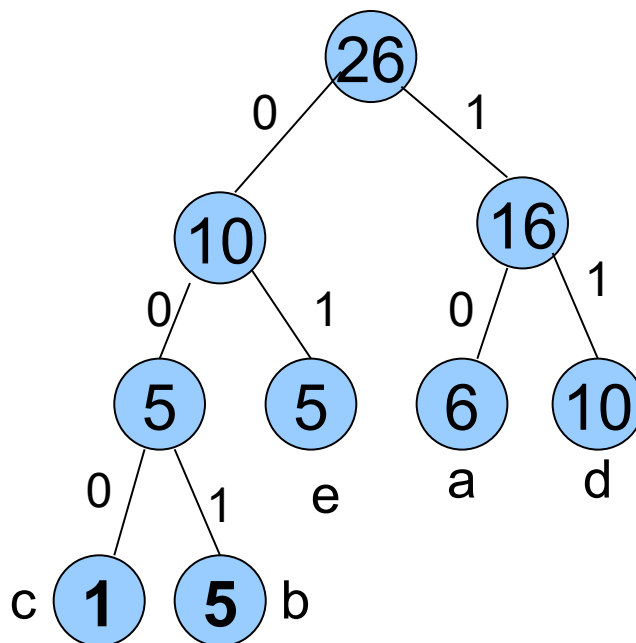


5.6.2 哈夫曼树的应用——哈夫曼编码

- 传送的电文为 'ABACCDA'
- (1) 定长编码： A,B,C,D 编码为 00,01,10,11
传输的长度为总长 15 位
- (2) 可变长编码 : A,B,C,D 编码为 0,00,1,01
传输的长度为总长 9 位，但却无法译码
- (3) 使用可变长编码的前缀编码，使每个符号
唯一编码，用哈夫曼树构造最短的前缀编码。

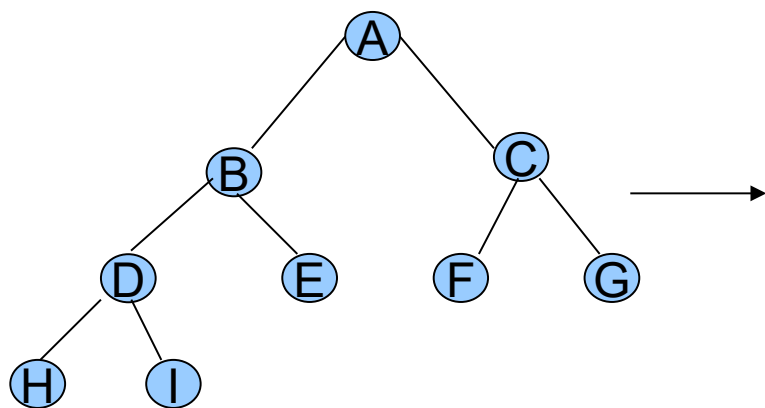
5.6.2 哈夫曼树的应用——哈夫曼编码

- 例：字符集 a,b,c,d,e 字符出现的次数为 6,5,1,10,5，请对它们进行哈夫曼编码



5.6.3 哈夫曼算法的静态实现

1. 二叉树的静态链表实现

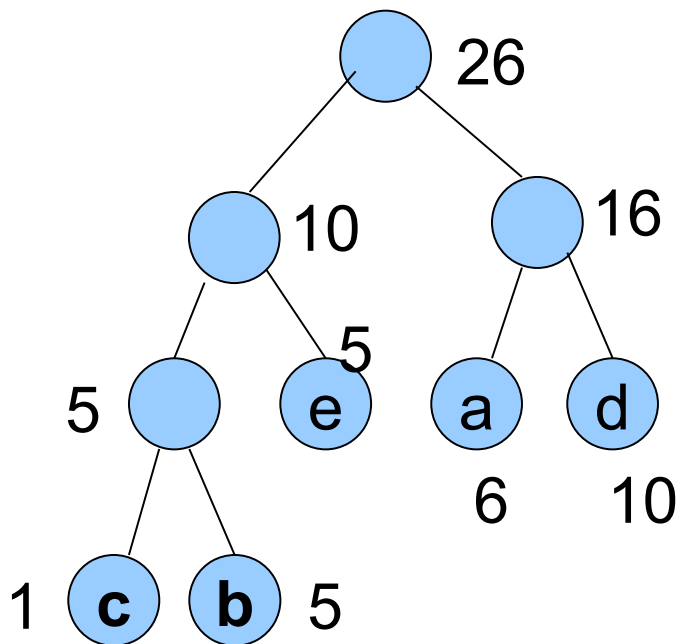


LCHILD RCHILD PARENT

0	A	1	2	-1
1	B	3	5	0
2	C	5	6	0
3	D	7	8	1
5	E	-1	-1	1
6	...			
7	H	-1	-1	3
8	I	-1	-1	3

2. 哈夫曼树的静态实现

哈夫曼树的静态存储结构



		左	右	双亲	权
0	c	-1	-1	5	1
1	b	-1	-1	5	5
2	e	-1	-1	6	5
3	a	-1	-1	7	6
5	d	-1	-1	7	10
5		1	5	6	5
6		5	2	8	10
7		3	5	8	16
8		6	7	-1	26

2. 哈夫曼树的静态实现

哈夫曼树的类型定义

```
#define maxnodenumber 100
typedef struct
{int weight;
 int parent,lchild,rchild;
}htnode; /* 定义结点类型 */
```

定义哈夫曼树类型

```
typedef htnode *huffmantree;
```

定义哈夫曼树的存储区域

```
htnode ht[maxnodenumber];
```

		左	右	双亲	权
0	c	-1	-1	5	1
1	b	-1	-1	5	5
2	e	-1	-1	6	5
3	a	-1	-1	7	6
5	d	-1	-1	7	10
5		1	5	6	5
6		5	2	8	10
7		3	5	8	16
8		6	7	-1	26

2. 哈夫曼树的静态实现

哈夫曼树的创建过程

例： a,b,c,d,e: 6,5,1,10,5

```
htnode ht[maxnodenumber];
huffmantree crthuffmantree(int n)
{int i, j, m1, m2, k1, k2;
/*1. 初始化数组*/
for (i=0; i<2*n-1; i++)
{ht[i].weight=0;
ht[i].parent= -1;
ht[i].lchild= -1;
ht[i].rchild= -1;
}
/*输入n个结点的权重 */
for (i=0; i<n; i++)
scanf( "%d" , &ht[i].weight);

for (i=0; i<n-1; i++)
{m1=maxvalue;
```

	左	右	双亲	权
0	-1	-1	7	6
1	-1	-1	5	5
2	-1	-1	5	1
3	-1	-1	7	10
5	-1	-1	6	5
5	2	1	6	5
6	5	5	8	10
7	0	3	8	16
8	6	7	-1	26

3. 哈夫曼编码的实现

	左	右	双亲	权
0	-1	-1	7	6
1	-1	-1	5	5
2	-1	-1	5	1
3	-1	-1	7	10
5	-1	-1	6	5
5	2	1	6	5
6	5	5	8	10
7	0	3	8	16
8	6	7	-1	26

- 思考：如何从已知的哈夫曼树得出哈夫曼编码
- 提示：从叶子结点出发找双亲，如果是双亲的做孩子则编码 0，右孩子编码 1。每个叶子结点编码用一维数组存储。N 个叶子结点则需要 n 个相同的一维数组

3. 哈夫曼编码的实现（续）

- 对于每个叶子结点的哈夫曼编码可以考虑用一个一维数组 `bit` 从后向前依次保存所求得的各位编码值，并用 `start` 记住编码在数组 `bit` 中的开始位置。
- 由此可做如下的类型定义：

```
#define maxbit 10 // 定义编码最大长度
typedef struct      // 定义保存一个叶子节点的结构
{int bit[maxbit]; /* 用一维数组保存一个叶子结
                  点编码 */
  int start;        // 开始位置域
}hcodetype;
Hcodetype cd[maxnodenum]; /* 定义编码数组，保存所有
叶子结点编码，具体使用过程中，只用到 n 个长度
```


3. 哈夫曼编码的实现（续 1）

	左	右	双亲	权
0	-1	-1	7	6
1	-1	-1	5	5
2	-1	-1	5	1
3	-1	-1	7	10
5	-1	-1	6	5
5	2	1	6	5
6	5	5	8	10
7	0	3	8	16
8	6	7	-1	26

编码

Cd[5]								start	
							1	0	8
						0	0	1	7
						0	0	0	7
							1	1	8
							0	1	8

哈夫曼树的编码结果

哈夫曼树

3. 哈夫曼编码的实现（续）

```

hcodetype cd[maxnodenum];
void gethuffmancode(int n)
{
    int i, j, c, p;
    for(i=0; i<n; i++) // 对 n 个叶子结点逐一求编码
    {
        c=i; j=maxbit;
        do{ j--; p=ht[c].parent;
            if(ht[p].lchild==c) /* 如果 c 是 p 的左孩子 */
                cd[i].bit[j]=0;
            else
                cd[i].bit[j]=1;
            c=p;
        } while(ht[p].parent != -1);
        cd[i].start=j;
    }
    for(i=0; i<n; i++) // 输出 n 个字符编码
    {
        for(j=cd[i].start; j<maxbit; j++)
            printf("%d", cd[i].bit[j]);
        printf("\n");
    }
}
    
```

	左	右	双亲	权
0	-1	-1	7	6
1	-1	-1	5	5
2	-1	-1	5	1
3	-1	-1	7	10
5	-1	-1	6	5
5	2	1	6	5
6	5	5	8	10
7	0	3	8	16
8	6	7	-1	26

小结

- 树的基本概念
- 二叉树
- 二叉树的遍历
- 线索二叉树
- 树和森林
- 哈夫曼树